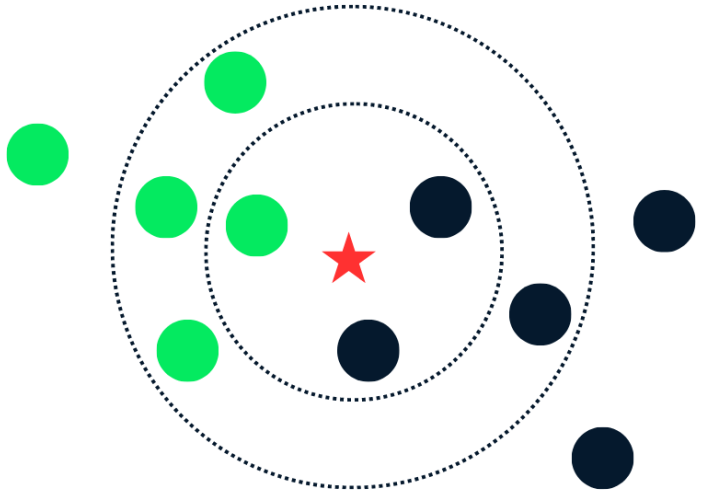
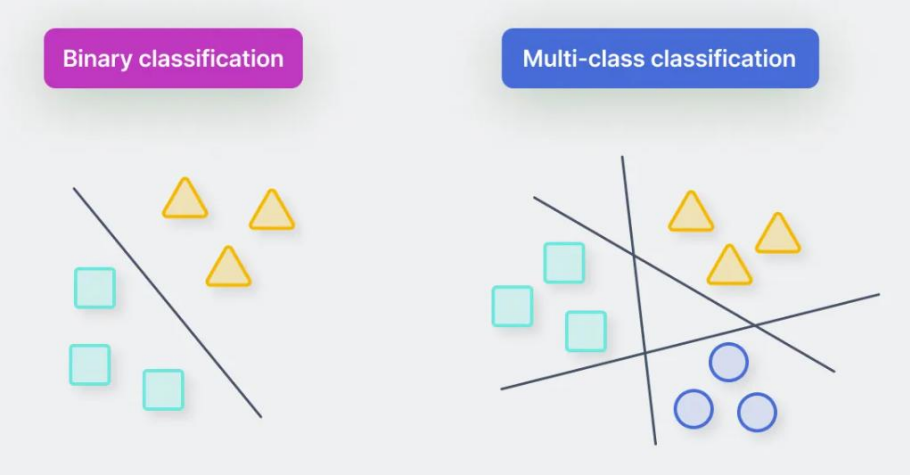
K-Nearest Neighbour (*k*NN) Classifier



Outline

- Classification
- K-Nearest Neighbour (kNN) Classifier

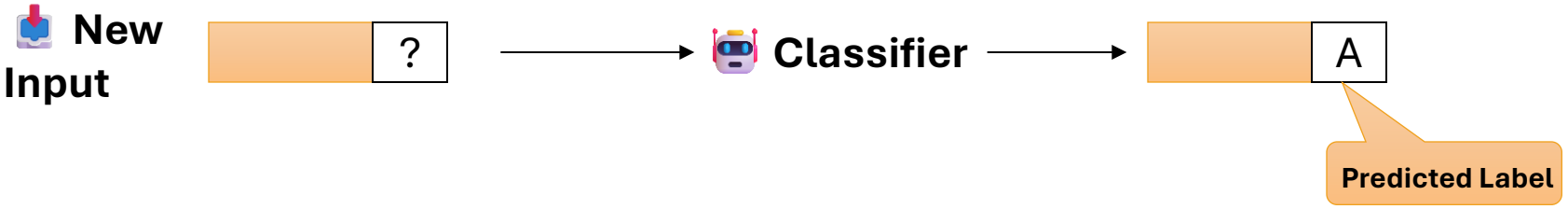
Classification



Classification Models

- 🎯 **Goal**: Predict the **class label** of an input instance using supervised learning
- Supervised learning two-stage process:
  - Given a **labeled** dataset containing features and their class labels, use a ⚙️ Learning Algorithm to train a 🤖 **Classifier** that maps features to class labels

  - Given **a new input** with unknown label, use the trained 🤖 **Classifier** to assign the most **likely class**



Classification Examples

- 📧 **Spam Detection (Binary)**
  Classifies emails as *spam* or *not spam* by learning patterns from text features and sender metadata.
- 💬 **Sentiment Analysis (Multi-class)**
  Predicts whether reviews or social-media posts express *positive*, *negative*, or *neutral* sentiment.
- 🩺 **Medical Diagnosis (Binary)**
  Uses patient symptoms and clinical history to classify the likelihood of a specific condition.
- 💳 **Credit Risk Assessment (Multi-class)**
  Categorizes applicants into *low*, *medium*, or *high-risk* using features such as credit score, income, and debt ratios.
- 🖼️ **Image Recognition (Multi-class)**
  Classifies images into categories (e.g., *cat*, *dog*, *bird*) based on learned visual features.

Classification Algorithms

📍 **K-Nearest Neighbors (KNN)**
Instance-based classifier that predicts using distance to nearby examples.

🌳 **Decision Trees &** 🎄 **Random Forests**
Tree-structured models that learn hierarchical decision rules.

📊 **Logistic Regression**
Linear classifier that estimates class probabilities using a logistic function.

📦 **Naive Bayes**
Probabilistic model assuming conditional independence between features.

🧱 **Support Vector Machine (SVM)**
Maximizes class-separation margin using an optimal hyperplane.

🧠 **Artificial Neural Networks (ANN)**
Multi-layer networks capable of learning complex nonlinear decision boundaries.

ML Metrics: Key Factors for Model Quality

📈 **Accuracy**
Proportion of correctly classified instances out of all predictions made

⚡ **Performance**

- Training time required to build the model
- Inference time required to generate predictions

🔍 **Interpretability**

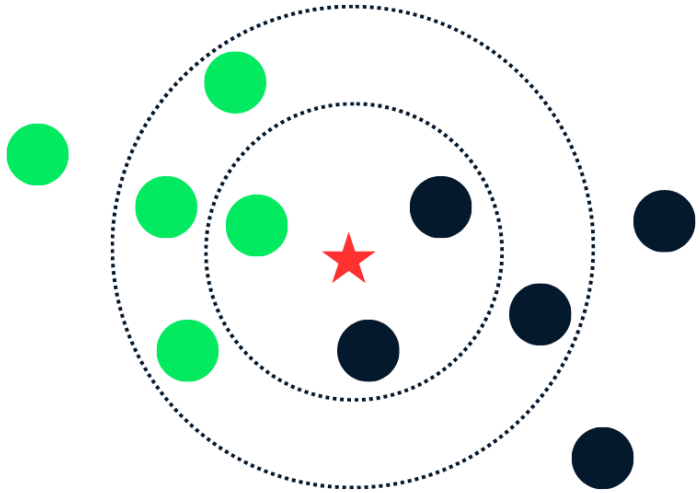
- How easily model decisions can be understood

🛡️ **Robustness**
Ability to handle noise, missing values, and imperfect data

📉 **Scalability**
Capability to operate efficiently on large datasets

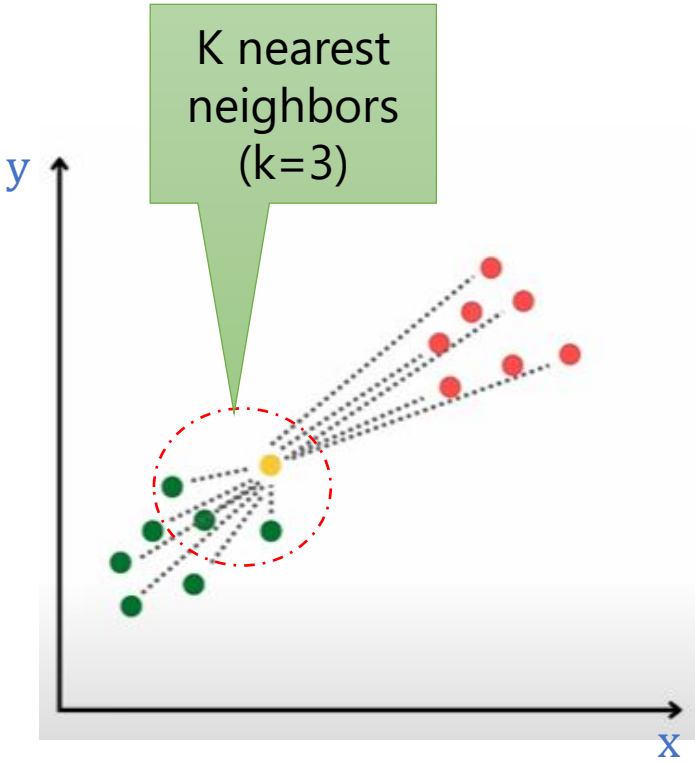
📦 **Model Complexity**
Size and compactness of the learned representation (e.g., depth of trees, number of rules)

K-Nearest Neighbour (*k*NN) Classifier



kNN Algorithm steps

- kNN predicts the label of a new instance based on the labels of the most **similar** training examples.
- Given a data point that we want to classify:
  1. **Compute the distance** between the new instance and all training examples
  2. **Locate the k nearest neighbors** with the smallest distances
  3. For classification: Assign the **majority class** among the k neighbors
  3. For regression: Compute the **average value** of the k neighbors



kNN Key Characteristics (1 of 2)

- 🐌 **Lazy (Instance-Based) Learner**:
  - It does not build an **explicit model** during training
  - All computation happens **at prediction time** using stored training instances

  👉 Ideal when data changes frequently because no retraining is required
- 📊 **Non-parametric** Algorithm:

Makes no assumptions about the underlying data distribution

- 🔍 **Similarity-based** learning:
  - Assumes that similar instances have similar labels
  - Classifies new examples based on their closest neighbors in feature space

kNN Key Characteristics (2 of 2)

- 🧩 **Simple to Implement**
  Easy to understand and apply; often a strong baseline method.

- 📏 **Requires Feature Scaling**
  Distance can be dominated by features with large numeric ranges (e.g., height 1.5–1.8 m, weight 90–300 lb, income 10k–500k QR)

- ⚙️ **Minimal Tuning**
  Only **k** and the **distance metric**

- 👍 **Surprisingly Effective**
  Often performs well—worth trying first on many problems.

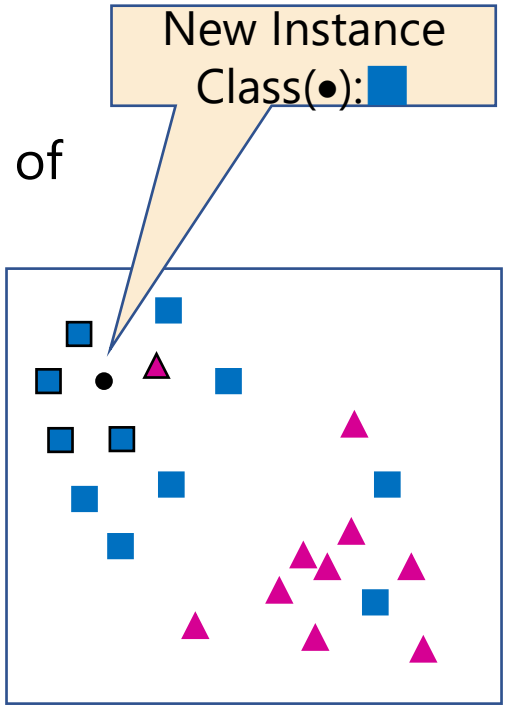
- ⏳ **Expensive at Prediction Time**
  Classifying a new instance requires scanning the **entire dataset**, which becomes 🐢 slow for large datasets.

- 📉 **Performs poorly in high dimensions** since accuracy degrades as the number of features grows.

- 📏 **Sensitive to the distance metric**; different metrics can produce different results

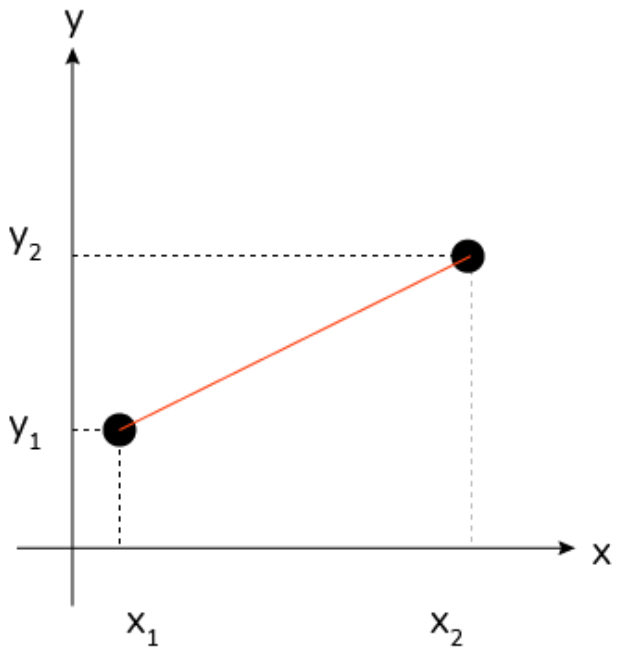
Requirements & Steps

- **Requirements:**
  - The set of training instances
  - Distance Metric to compute distance between instances
  - The value of the hyperparameter *k*, i.e., the number of nearest neighbors to consider
- **To predict an unknown instance:**
  - Compute distances to all training instances
  - Locate ***k*** nearest neighbors
  - **Classify using majority vote** among the k neighbors
  - In the case of a tie:
    - Use odd *k*
    - Choose the class with the **closest neighbor**
    - Pick the class dominant in the entire dataset
    - Decrease *k* by 1 until you break the tie



Euclidean Distance

- **Euclidean Distance** is a measure of the **straight-line distance** between two points $(x_1, y_1)(x_2, y_2)$ in a two-dimensional space



It's derived from the **Pythagorean** theorem

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Numerical measure of **Dissimilarity/Distance**
  - How **different** are two data objects
  - Lower when objects are more alike

Euclidean Distance for data objects with multiple features

- When dealing with instances (data objects) with multiple $\mathbf{n}$ features, the Euclidean distance between two instances $\mathbf{p}$ and $\mathbf{q}$ with features ($p_1, p_2, \ldots p_n$) and ($q_1, q_2, \ldots q_n$) respectively, is given by:
  - the square root of the sum of the squared differences between corresponding feature values

$$\text{Distance} = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

- p_i and q_i represent the values of the i^{th} feature for data objects $\mathbf{p}$ and $\mathbf{q}$ respectively
- $\mathbf{n}$ is the number of features

slides\knn.xlsx

Distance Metrics

- Euclidean distance: $d_{Eucl}(a,b) = \sqrt{\sum_{k=1}^{m}(a_k - b_k)^2}$

  Measures straight-line geometric distance

- Manhattan distance: $d_{Manh}(a,b) = \sum_{k=1}^{m}|a_k - b_k|$

  Sum of absolute differences

- Minkowski distance: $d_{Mink}(a,b) = \sqrt[p]{\sum_{k=1}^{m}|a_k - b_k|^p}$

  Family of metrics

  - When p = 1, the Minkowski distance is the Manhanttan distance
  - When p = 2, the Minkowski distance is the Euclidean distance
  - Although there are infinite number of Minkowski-based distance metrics to choose from, Euclidean distance and Manhattan distance are the most used ones

When to Use Each Distance Metric

📐 **Euclidean Distance (p = 2)**

- Features are continuous and vary smoothly together
- Data tends to form rounded / spherical clusters
- Low to medium-dimensional datasets
- Emphasis on large feature differences

Typical for: image pixels, sensor readings (e.g., temperature, humidity, pressure readings change jointly), geometric data (e.g., coordinates (x, y, z) naturally follow straight-line distance).

📏 **Manhattan Distance (p = 1)**

- Features vary independently, not in combined directions (moves along grid-like paths (like city blocks)) e.g., login count, app error count
- More robust to noise and outliers
- Works better for high-dimensional /sparse datasets

Typical for: text embeddings (represent text as numerical vectors), one-hot encoded categorical features such gender, payment methods.

📦 **Minkowski (p > 2 or fractional p)**

- You want to experiment with different geometries (e.g., p = 3: more sensitive to large feature differences than Euclidean)
- You need a metric between Manhattan and Euclidean (e.g., p = 1.5)
- You want to tune p as a hyperparameter (rare but sometimes useful)

Similarity Between Binary Vectors using Simple Matching Coefficient (SMC)

- SMC quantifies the similarity between two **binary** vectors p & q (e.g., gender, yes/no, presence/absence)
  - **SMC ranges from 0** (no similarity) **to 1** (perfect similarity, all matches)

$$SMC = \frac{M_{11} + M_{00}}{M_{11} + M_{00} + M_{01} + M_{10}}$$

M_{11}: Counts positions where **both vectors have 1**
M_{00}: Counts positions where **both vectors have 0**
M_{01}: Counts positions where **p = 0** and **q = 1**
M_{10}: Counts positions where **p = 1** and **q = 0**

- In the case of **multiple features**, the formula is applied to each feature individually, and then averaged across all n features:

$$SMC = \frac{1}{n} \sum_{i=1}^{n} \frac{M_{11}^{(i)} + M_{00}^{(i)}}{M_{11}^{(i)} + M_{00}^{(i)} + M_{01}^{(i)} + M_{10}^{(i)}}$$

Similarity Between Binary Vectors using Jaccard Coefficient

- Jaccard Coefficients is used to quantify the similarity between two **binary** vectors

$$J = \frac{number\ of\ 11\ matches}{number\ of\ not-both-zero\ attribute\ values}$$

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}}$$

- SMC / Jaccard ranges from 0 to 1, where:
  - 0 indicates no similarity between the instances
  - 1 indicates perfect similarity between the instances
  - Higher SMC values suggest greater similarity between the instances, while lower values suggest greater dissimilarity

SMC versus Jaccard - Example

p = 1, 0, 1, 0, 1

q = 1, 1, 0, 0, 1

M_{11} = 2 (the number of attributes where p is 1 and q is 1)

M_{00} = 1 (the number of attributes where p is 0 and q is 0)

M_{01} = 1 (the number of attributes where p is 0 and q is 0)

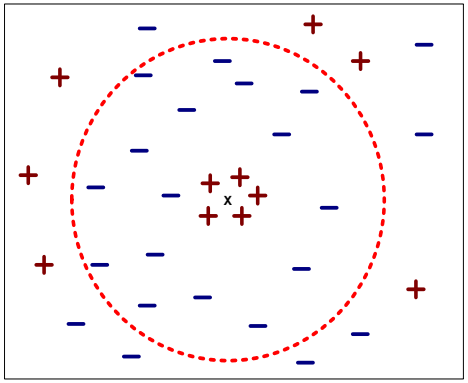
M_{10} = 1 (the number of attributes where p is 1 and q is 0)

$$\text{SMC} = \frac{M_{11} + M_{00}}{M_{11} + M_{00} + M_{01} + M_{10}} = \frac{2 + 1}{2 + 1 + 1 + 1} = \mathbf{0.6}$$

$$\text{J} = \frac{M_{11}}{M_{01} + M_{10} + M_{11}} = \frac{2}{2 + 1 + 1} = \mathbf{0.5}$$

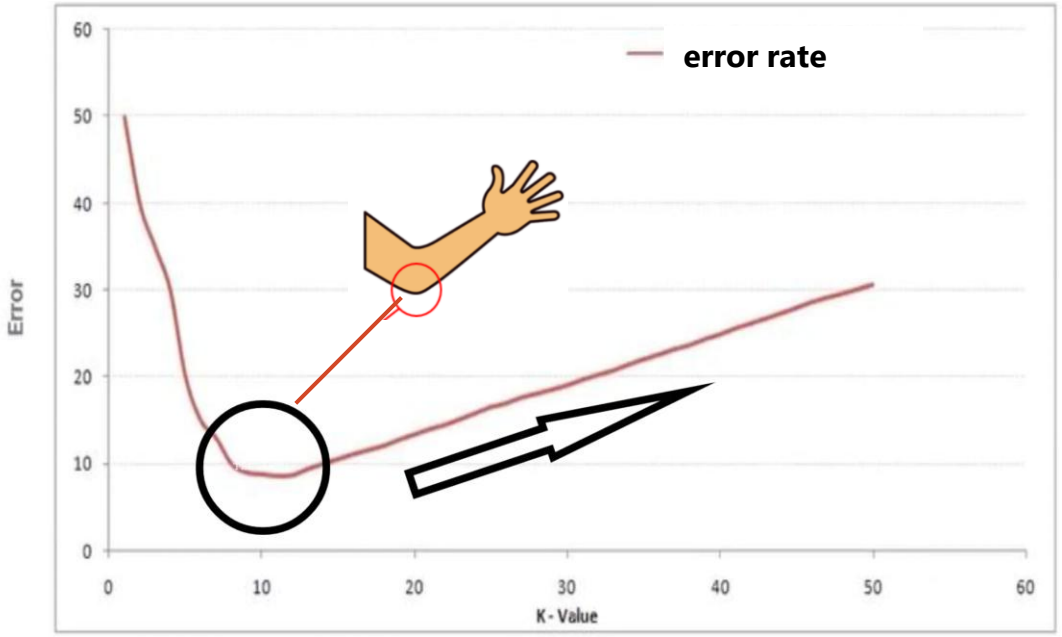
Choosing the value of k

- **k too small → Overfitting = poor generalization (High Variance)**
  Model becomes too sensitive to noise; predictions change a lot with small changes in the data
- **k too large → Underfitting (High Bias)**
  Neighborhood becomes too broad; the model oversimplifies patterns and mixes points from different classes
- **Goal:**
  Select a k that balances **bias and variance** for optimal classification performance



The Elbow Method

1. Train kNN with a range of k values
2. Compute the error rate for each k
3. Plot error vs. k



- Identify the "elbow" point on the curve, where the **error rate stops decreasing sharply** and starts to level off
- **Choose k at the elbow point** as the optimal number of neighbors
- This elbow indicates the **best trade-off** between overfitting (small k) and underfitting (large k).